

FINAL PROJECT

Forecasting Competitive Football

Pre-Match & In-Play Prediction from Raw Event Data

Machine Learning Course | Department of Computer Science

GRADING EMPHASIS

Component	Weight	What Examiners Focus On
Data Integration & Feature Pipelines	30%	Correct multi-file joins, leakage-proofing, dual pipeline design
Paper Reimplementations (P1 + P2)	20%	Fidelity to methodology, honest failure reporting
Three Models & Comparative Analysis	20%	All three deliverables working, calibration quality
Written Report	15%	Rigour, narrative clarity, honest limitations
Defences (2 mid + final)	15%	Pipeline reasoning, modeling choices, live SHAP walkthrough

Submission Deadline: See Course Portal | Groups of 1-2 | All code must be reproducible

TABLE OF CONTENTS

1. Introduction & Problem Framing.....	3
2. Dataset: StatsBomb Open Data.....	5
3. Data Leakage Rules.....	8
4. Finding Your Two Papers.....	10
5. Models & Baseline Comparison.....	12
6. Full Pipeline Flowchart.....	14
7. Imbalance, Resampling & Explainability.....	16
8. Report Structure.....	18
9. Submission Checklist.....	19
10. Defence Milestones.....	20
11. Bonus: Real-Time Prediction Service.....	22
12. References.....	23

1. INTRODUCTION & PROBLEM FRAMING

Football (soccer) is the world's most-watched sport, and modern football analytics runs on event data: a complete, timestamped log of everything that happens on the pitch. Every pass, carry, pressure, duel, shot, foul, and substitution is recorded with its pitch coordinates, the player and team involved, and rich contextual detail. A single match produces close to four thousand such events. This is the raw material from which professional clubs, bookmakers, and broadcasters build their models, and it is the raw material you will work with in this project.

In real deployment, football forecasting happens at two distinct moments. Before kick-off, a pre-match model uses everything known about the two teams from their history (form, style, squad strength, venue) to publish outcome probabilities and an expected scoreline; this is how opening bookmaker prices and pre-game analytics are produced. Then, once the ball is rolling, an in-play model takes over: it watches the event stream and continuously re-estimates the same quantities as goals, red cards, injuries, and momentum shifts arrive. This is the engine behind live betting markets and the win-probability graphics that update on screen during a broadcast. The two models answer the same questions but live on completely different data pipelines, and building both, honestly and without leakage, is the heart of this project.

You will work with a single data source, the StatsBomb Open Data repository, and take it from raw nested JSON files all the way to three working predictive models. Nothing is pre-tabulated for you: integrating the competition, match, lineup, and event files into one coherent relational structure, defining what a training example even is for each task, and labeling it correctly is itself a major graded challenge. Expect the data engineering to consume as much effort as the modeling, exactly as it does in industry.

Because real forecasting is judged on the quality of its probabilities, not merely on whether the single most likely outcome occurred, this project weights calibration and proper scoring rules heavily. A model that confidently predicts a home win and is wrong must be punished more than one that expressed appropriate uncertainty. And because a forecast that cannot be explained cannot be trusted, SHAP-based explanation of individual predictions is a required deliverable, culminating in a live walkthrough at the final defence.

The Three Model Deliverables

You must design, formally define, and deliver three models. For each one, your report must state precisely what one input example is, what the output is, and how the training set is constructed, before any modeling begins. This problem-definition step is graded.

- **Model 1 -- Pre-Match Outcome Classification (Task C).** Input: one feature vector per match, built exclusively from information available before kick-off (aggregates of the two teams' prior matches, venue, rest days). Output: a calibrated probability distribution over Home Win / Draw / Away Win. Headline metric: Ranked Probability Score (RPS); secondary: Log-Loss, Brier Score, ECE with reliability diagrams.
- **Model 2 -- Pre-Match Goal-Margin Regression (Task R).** Same pre-match input as Model 1. Output: the signed goal difference (home minus away), clipped to $[-5, +5]$. Metrics: MAE, RMSE, and correlation with actual margins. You must also analyse whether Model 2's margin can be converted into useful Model 1 probabilities.
- **Model 3 -- In-Play (Live) Prediction (Task L).** Both predictions re-estimated during the match. Input: a snapshot of the game state at time t , combining the pre-match feature vector with features computed from the events of THIS match up to and including time t only: current score, man-advantage, event counts and rates in a recent window, momentum indicators. Output: updated

outcome probabilities and an updated expected final margin. One match therefore generates many training examples (for instance one snapshot every five minutes), and this requires a second, separate feature pipeline. Evaluation must show performance as a function of match minute: as information arrives, Model 3 must increasingly outperform its own pre-match prior, and its probabilities must converge toward certainty as full time approaches.

Model 3 is deliberately the hardest deliverable. It forces you to define a snapshotting scheme, to prevent within-match leakage (no event after time t may influence the snapshot at t), to keep all snapshots of one match on one side of the train/test boundary, and to reason about calibration per game phase. It is also where the project touches the real world most directly: this is precisely the model class running behind every live win-probability product.

2. DATASET: STATSBOMB OPEN DATA

The single data source for this project is the StatsBomb Open Data repository, the free public release of professional-grade event data from Hudl StatsBomb. It is the highest-quality open football dataset in existence and the de-facto standard in football analytics research and education. All figures below were verified directly against the repository.

StatsBomb Open Data -- Verified Specification	
Source	github.com/statsbomb/open-data (free, JSON files). Python client: statsbombpy. Non-commercial use with attribution.
Scale	80 competition-season entries across roughly 25 competitions, men's and women's, club and international.
Full league seasons	La Liga: 18 seasons (2004/05-2020/21 plus 1973/74). Premier League 2015/16 (380 matches) and 2003/04. Bundesliga 2015/16 and 2023/24. Serie A 2015/16 and 1986/87. Ligue 1 2015/16, 2021/22, 2022/23.
Tournaments	FIFA World Cup (8 editions incl. 2018, 2022), UEFA Euro 2020 and 2024, Copa America 2024, Women's World Cup 2019 and 2023, UEFA Women's Euro 2022 and 2025, African Cup of Nations 2023, Champions League finals across 18 seasons.
Event granularity	Close to 4,000 events per match (verified: 3,947 in a sample La Liga match). Passes, ball receipts, carries, pressures, duels, interceptions, dribbles, shots, fouls, goalkeeper actions, and more.
Event fields	Every event carries a type, period, minute, second, timestamp, possession index, team, player, position, play pattern, pitch location on a standardized 120 x 80 coordinate system, plus type-specific detail (pass height, body part, shot outcome, freeze-frame of all players at shot moments).
360 data	For selected matches, StatsBomb 360 frames record the positions of all visible players around each event.
File structure	See the repository map below. Everything is keyed by competition_id, season_id, and match_id.

2.1 Repository Structure

File / Folder	Contents	Keys
competitions.json	Master list of all available competition-season pairs	competition_id, season_id
matches/{comp_id}/{season_id}.json	All matches of one season: teams, date, kick-off, final score, stage, referee	match_id
events/{match_id}.json	The full ordered event stream of one match (about 4,000 rows)	event id, index, timestamp
lineups/{match_id}.json	Both squads: players, jersey numbers, positions played	player_id, team_id
three-sixty/{match_id}.json	Player-position freeze frames (selected matches only)	event uuid

2.2 The Integration Challenge

This dataset is deliberately not a flat table. Turning it into modeling-ready data is the first graded deliverable of the project, and it is substantial. You must design and defend a relational integration: parse the JSON tree, join the five file families on their shared keys, resolve player and team identities across seasons and competitions, and materialise clean tables from which both feature pipelines can be built. You must then define the problem formally: what is one example, what is its label, and which rows of which files are allowed to contribute to it. Expect and document real-world imperfections: 360 files exist only for some matches, some competitions are single tournaments while others are full league seasons, and event schemas carry optional fields that appear only for certain event types.

- **Competition selection:** you must justify which competitions you train on. Full league seasons (La Liga's 18 seasons are the backbone) give volume and temporal depth for the pre-match models; tournaments add variety but little history. Mixing them naively creates distribution shift that you must measure, not ignore.
- **Labeling:** derive Win / Draw / Loss and the signed margin from home_score and away_score in the matches files. For Model 3, the label of every snapshot of a match is that match's final outcome, which creates strong within-match correlation that your evaluation design must respect.
- **Feature ideas from events:** shot counts and shot locations (an xG proxy), pressure intensity, passing volume and completion by zone, carries into the final third, set-piece counts, possession share, defensive actions. Aggregated over a team's prior matches these become pre-match features; computed over a rolling window inside the current match they become in-play features.

2.3 Odds Source & Market Baseline: Football-Data.co.uk

StatsBomb gives you the events; it does not give you the market. For that you must integrate a second source: Football-Data.co.uk, which publishes free per-season CSV files for every division of the major European leagues (for England, football-data.co.uk/englandm.php; equivalent pages exist for Spain, Germany, Italy, France, and others). Each row is one match carrying full-time and half-time results, match statistics (shots, shots on target, fouls, corners, cards), and pre-match bookmaker odds for the match result, total goals, and the Asian Handicap, from several bookmakers. Odds coverage runs from season 2000/01 onward for the top divisions.

Football-Data.co.uk -- What You Must Do With It	
Download	Pick the league pages matching your chosen StatsBomb competitions (e.g. spainm.php for La Liga, englandm.php for the Premier League) and download every season CSV that overlaps your data.
Tag to match_id	Join each CSV row to its StatsBomb match_id. There is no shared key: you must match on (date, home team, away team) and resolve team-name aliases between the two sources (e.g. 'Man United' vs 'Manchester United'). Document the mapping and its failure cases.
De-vig	Convert the 1X2 decimal odds of a chosen bookmaker (or an average) into probabilities and remove the over-round by normalising the three to sum to one (Section 3.2).
Publish the baseline	The de-vigged market probability is THE main baseline of this project. Every classification model, pre-match and in-play at kick-off, is judged first and foremost against it.
Test set discipline	Your test set may only contain matches that were successfully tagged with odds, so the model-vs-market comparison is computed on the exact same matches. Report the tagging coverage rate per season and list every excluded match with the reason.

The tagging step is graded as part of the integration deliverable and is examined at Mid Defence 1. A model score that cannot be placed next to the market number for the same matches is scientifically meaningless in this domain, which is why the test-set creation pipeline matters as much as the models themselves.

3. DATA LEAKAGE RULES

Data leakage is the single most common source of invalid results in sports prediction, and event data multiplies the opportunities to leak. Leakage occurs whenever information that would not be available at prediction time influences the model. For the pre-match models the prediction time is kick-off; for the in-play model it is the snapshot time t . The rules below are absolute, and each is explained so you understand not only what is forbidden but why.

LEAKAGE CHECKLIST -- Every item must be YES before submission

Are all pre-match features computed strictly from matches that finished BEFORE the target match's kick-off?

For every in-play snapshot at time t , is every feature computed ONLY from events with timestamp $\leq t$ in that match?

Do all snapshots of one match live entirely on one side of the train/validation/test boundary?

Was every resampling method (SMOTE and variants) fit ONLY on training folds, never before the split?

Is every scaler, encoder, and imputer fit on training data only, then applied unchanged to validation and test?

Is the final outcome (or any post-snapshot event, like the eventual shot count) absent from every snapshot's features?

Are bookmaker odds used EITHER as the market baseline OR as a declared feature, never both in the same experiment?

Was the odds-to-match_id tagging done on pre-match identity only (date, teams), never on scores or statistics?

3.1 Why Each Rule Matters

- **Pre-kickoff features only (Models 1 and 2).** Form and event-rate aggregates must summarise the past. A rolling average that accidentally includes the target match leaks the very outcome you are predicting and produces a model that is unbeatable on paper and useless in reality.
- **The time- t boundary (Model 3).** This is the sharpest trap in the project. Event files contain the whole match; when you snapshot at minute 30, a single careless join can pull in the 70th-minute goal. Sort events by period, timestamp, and index, cut strictly at t , and unit-test the cut with an assertion that no contributing event postdates the snapshot.
- **Match-level splitting.** Snapshots from the same match share a label and are heavily correlated. If minute-30 of a match sits in training and minute-60 of the same match sits in test, the test score is fiction. Split by match (and chronologically by season), then let snapshots inherit their parent match's side.
- **Resample inside folds.** Fitting SMOTE on the full dataset lets synthetic minority points be built from test-set neighbours, so information crosses the split. On temporal data it is worse: synthetic samples can be interpolated from future matches. Use the imbalanced-learn Pipeline so resampling is confined to training folds automatically.

- **Fit transforms on training data only.** A scaler or imputer fit on the whole dataset has already seen the test distribution. Fit on train, transform validation and test with those frozen parameters, so the evaluation reflects genuine generalisation.
- **Never tag odds using outcomes.** The Football-Data.co.uk join must use only pre-match identity (date, home team, away team). Matching on scores or match statistics silently guarantees the join is correct only when the result agrees, which biases the tagged sample.

3.2 The De-Vigged Market Baseline

Bookmaker decimal odds embed a margin, the over-round or vig, that makes the implied probabilities sum to more than one. To recover fair probabilities, convert each of the three 1X2 odds to its raw implied probability (one divided by the decimal odd) and normalise the three so they sum to exactly one. This de-vigged market probability, computed from your tagged Football-Data.co.uk odds, is the mandatory main baseline of the project: every classification model is reported alongside it on the identical set of tagged test matches. Beating the market consistently is a very high bar, and honest, well-analysed failure to beat it is worth more marks than an unexplained score that merely looks good. Nothing about this baseline removes any other required model: the Dummy prior remains the sanity floor, the market is the ceiling to attack, and your models live between them.

4. FINDING YOUR TWO PAPERS

You must find and reimplement two published papers from scratch. Paper P1 is a data paper: it may address any part of how the data is represented, generated, sampled, or engineered. Paper P2 is a model paper: it must introduce a predictive machine-learning model, not a deep neural network, that can handle both a classification and a regression task. Both papers must clear the hard filters below. Your paper choices are examined at the two mid defences (Section 10): P1 at Defence 1 together with your data pipeline, and P2 at Defence 2 together with your modeling approach.

HARD FILTERS -- Both papers must pass all four

Published in 2022 or later (use the publication date, not the preprint date).

Published in a Q1-ranked venue: verify journals via Scimago or JCR; verify conferences via CORE at rank A* or A.

The method must be directly applicable to this dataset and your chosen tasks (no unrelated-domain papers).

TA sign-off obtained BEFORE implementation begins, at or before the corresponding mid defence.

4.1 Paper P1 -- The Data Paper

P1 can be about anything that improves the data side of the problem and demonstrably fits this task. You are free to choose the sub-topic, but your choice must be justified against the actual imperfections of event data: class imbalance, within-match correlation of snapshots, sparse rare events, and distribution shift across competitions and eras. Acceptable directions include, but are not limited to, the following.

- **Data representation:** feature construction from event streams, temporal or rolling-window encodings, learned embeddings for teams or players, spatial-zone representations of the 120 x 80 pitch, or graph representations of passing networks.
- **Sampling and imbalance:** resampling schemes, cost-sensitive reweighting, informative under-sampling, or advanced minority-class strategies beyond plain SMOTE.
- **Synthetic data generation:** tabular synthesis (SMOTE or CTGAN family) to augment scarce situations, with honest validation that the synthetic data actually helps.
- **Negative sampling:** principled selection of negatives or contrastive pairs where a paper frames the problem in those terms.
- **Augmentation and data modelling:** noise injection, feature perturbation, missing-value modelling, or imputation strategies for matches lacking 360 data.
- **Data engineering:** leakage-aware feature pipelines, drift detection across competitions, or published feature-selection methodology.

4.2 Paper P2 -- The Model Paper

P2 must introduce a predictive machine-learning model that can be applied to both classification and regression, so that a single method can serve Models 1, 2, and 3. Deep learning is explicitly out of scope for P2: no neural networks, transformers, or deep sequence models. Choose a classical or modern non-deep ML model, for example a novel tree-ensemble or gradient-boosting variant, a probabilistic or Bayesian model, a kernel method, a calibrated ensemble, or a boosting extension aimed at calibration or uncertainty estimation. The model must be reimplemented following the paper rather than simply called from a library,

and your report appendix must contain a hand derivation of its core update or objective equation. Where the paper demonstrates only one task, adapting it cleanly to the other is part of your contribution.

4.3 How to Search Google Scholar

For P1 (data), try queries such as: event data feature representation sports analytics 2023, tabular data augmentation imbalanced classification, spatiotemporal feature engineering football, rolling window representation time series prediction. For P2 (model), try: gradient boosting calibration probabilistic prediction 2023, non-deep tabular model classification regression, Bayesian ensemble sports forecasting, kernel method calibrated probabilities. Use the Cited-by graph to move forwards from a strong survey and the reference list of each candidate to move backwards. Stop once you have papers that clear the hard filters and that you can realistically implement within the timeline.

5. MODELS & BASELINE COMPARISON

Train and evaluate every model in the table below. The Task column shows where each applies: C is pre-match classification (Model 1), R is pre-match margin regression (Model 2), and L is the in-play task (Model 3), where the same learner families are retrained on the snapshot feature set. Report the full metric suite for every combination. The point is not to crown a winner; it is to understand why performance differs across model families and what each one's failures reveal about the data. Detailed requirements follow on the next page.

Model	Python Library	Task(s)	Role & Notes
Dummy / Home-Advantage Prior	sklearn.dummy	C, R, L	Majority-class and home-win base-rate predictor. For L, also compare against the frozen pre-match prediction as a baseline.
Kernel SVM (SVC / SVR)	sklearn.svm	C, R	First kernel method. RBF SVC for classification, SVR for regression. Scale-sensitive; drives the $O(n^2)$ discussion.
Kernel Ridge Regression	sklearn.kernel_ridge (+ kernel_approximation)	R	Second kernel method. Use RBFsampler or Nystroem to scale; compare exact vs. approximate kernels.
Random Forest	sklearn.ensemble	C, R, L	Strong low-tuning ensemble. Variance-reduction reference; inspect feature importances against your engineered features.
Gradient Boosting (GBM)	sklearn.ensemble	C, R, L	Textbook boosting. Slower than XGBoost or LightGBM at scale; quantify the gap in the compute section.
XGBoost	xgboost	C, R, L	Regularised tree boosting; usual state of the art on tabular data. Tune depth, learning rate, subsampling.
LightGBM	lightgbm	C, R, L	Histogram-based boosting. The snapshot table for L is large; compare speed and memory against XGBoost and GBM on it.
De-Vigged Market Odds (Football-Data.co.uk)	derived from tagged odds	C	THE main baseline. De-vigged 1X2 probabilities on the tagged test matches; every classifier must be reported against it.
P1 Paper Method	per student	per paper	Your chosen data method (Section 4.1), reimplemented from scratch, applied wherever it fits.
P2 Paper Method	per student	C, R, L	Your chosen non-deep predictive model (Section 4.2), with a hand derivation in the appendix.

5.1 Comparison Requirements

The table on the previous page defines what to run. This page defines how the comparison must be conducted and reported. Treat each requirement as a gradable deliverable.

MANDATORY COMPARISON REQUIREMENTS

Installation: pip install scikit-learn xgboost lightgbm imbalanced-learn shap statsbombpy. Pin versions in requirements.txt.

Calibration: apply Platt Scaling or Isotonic Regression to every probabilistic classifier and report ECE before and after, with reliability diagrams. For Task L, report calibration per game phase (0-15', 15-30', ..., 75-90').

Market comparison: report every Task C model against the de-vigged Football-Data.co.uk baseline on the identical tagged test matches, and state the tagging coverage rate of that test set.

In-play evaluation: plot every Task L metric as a function of match minute, and always include the frozen pre-match prediction as the reference curve it must beat.

Compute cost: record wall-clock training time and peak memory for every model on the full training set, presented in one table (Section 8D). The snapshot table makes this non-trivial.

Kernel scaling: derive or cite the $O(n^2)$ space cost of exact kernel methods and demonstrate it empirically with a training-time vs. dataset-size plot.

No cherry-picking: report the complete metric suite for every model on every task, including where performance is poor.

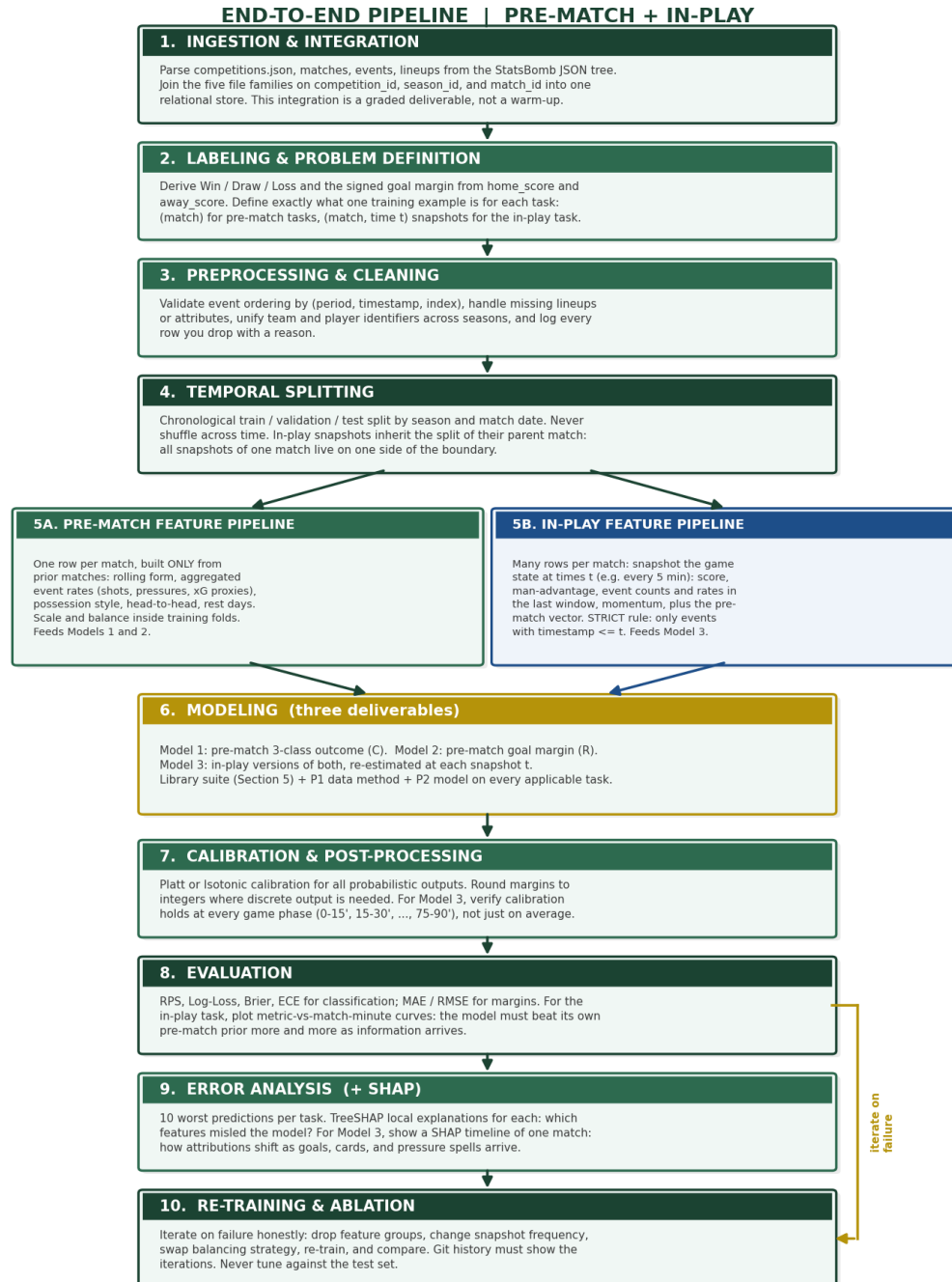
Fair tuning: give every model a comparable hyperparameter budget and describe the search protocol (grid, random, or Bayesian) explicitly.

5.2 What the Comparison Should Reveal

- **Family behaviour:** explain why boosted trees typically dominate tabular data, where kernel methods struggle at scale, and when the low-tuning Random Forest is already close to the best result.
- **Calibration vs. accuracy:** show that better arg-max accuracy does not imply better calibration, and argue which property matters more for a forecasting product.
- **Market gap:** quantify how far each model sits from the de-vigged market baseline, and hypothesise which signals the market prices in that your event-derived features miss.
- **Information value of the live stream:** quantify how much each in-play feature group (score state, man-advantage, recent event rates) improves Task L over the pre-match prior, and at which match phases.
- **Paper vs. library:** position your P1 and P2 reimplementations against the tuned library baselines and report the outcome honestly, including negative results.

6. FULL PIPELINE FLOWCHART

The diagram below is the complete end-to-end pipeline, from raw JSON ingestion through the split into the two parallel feature pipelines (pre-match and in-play), into the three model deliverables, and back through the iteration loop. Each stage lists its key design decisions and the pitfalls that most often cost marks. Your report must map every implementation choice to its stage in this diagram. The gold loop represents honest iteration after error analysis, never repeated tuning against the test set.



7. IMBALANCE, RESAMPLING & EXPLAINABILITY

Football outcomes are not uniformly distributed. Across large samples, home wins occur roughly 45 percent of the time, away wins around 28 percent, and draws around 27 percent, and the mix drifts by competition and era. The imbalance is milder than in fraud detection, but it is real, and for the in-play task it changes shape over the match: snapshots taken while a team leads by two goals are dominated by one eventual outcome. How you handle imbalance interacts with calibration in ways that must be studied rather than assumed.

7.1 Required Resampling Comparisons

Run the following on identical splits and compare them on both aggregate and per-class metrics, for the pre-match classification task at minimum.

- **Vanilla:** train on the raw class distribution with no resampling, as the honest baseline.
- **SMOTE:** synthetic minority oversampling via imbalanced-learn, fit inside training folds only.
- **BorderlineSMOTE:** concentrates synthetic samples near the decision boundary where errors cluster.
- **ADASYN:** adaptively synthesises more samples for the harder minority regions.
- **Class weights:** pass `class_weight='balanced'` to the estimators instead of resampling, and compare the trade-off.
- **Your P1 method:** if P1 addresses imbalance or data generation, include it as an additional arm of this study.

7.2 Evaluation Protocol for Imbalance

Compare every method using both aggregate scores (RPS, Log-Loss) and per-class precision, recall, and F1, and plot a reliability diagram for each. A recurring lesson is that improving minority-class recall through aggressive oversampling often degrades probability calibration, so this trade-off must be measured and discussed rather than ignored.

WARNING: resampling belongs inside training folds only

Fitting SMOTE on the full dataset before splitting is a form of data leakage and will be penalised.

Use the imbalanced-learn Pipeline (not sklearn's Pipeline) so resampling never touches validation or test data.

Never oversample the snapshot table across matches: synthetic snapshots interpolated between two different matches are physically meaningless and leak label information.

7.3 Explainability with SHAP

Wherever a model supports it, use SHAP (SHapley Additive exPlanations) values to explain predictions, and include this analysis in your report. SHAP assigns each feature a signed contribution to a specific prediction, grounded in cooperative game theory, so that the contributions for one prediction sum to the gap between that prediction and the model's average output. This turns an opaque score into an auditable statement about which features drove it. SHAP is also the centrepiece of your final defence (Section 10).

- **Tree models:** use TreeSHAP (the shap library's TreeExplainer) for Random Forest, GBM, XGBoost, and LightGBM. It is exact and fast on tree ensembles, which is why these models are the natural place to apply SHAP here.
- **Global view:** produce a SHAP summary (beeswarm) plot per model and task to rank features by overall impact, and compare it against built-in feature importances.
- **Local view:** produce SHAP force or waterfall plots for individual predictions, especially the worst ones from the error analysis, to show why the model was confidently wrong.
- **In-play SHAP timeline:** for at least one full match, plot how the SHAP attributions of Model 3 evolve snapshot by snapshot: watch score state take over from pre-match form after a goal, or man-advantage surge after a red card. This single figure demonstrates the entire project working end to end.

The purpose is not to produce plots for their own sake. SHAP is how you connect the error analysis to concrete feature behaviour: when a model fails on comebacks or on newly promoted teams, SHAP should reveal which features misled it, and that evidence should inform your re-training and ablation choices.

8. REPORT STRUCTURE

Your written report must follow the structure below. Page counts are guidance, not hard caps; depth and honesty of analysis are weighted above length.

Section	Title	Suggested Pages
1	Introduction & Problem Framing (formal definition of all three tasks)	1-2
2	Dataset, Integration Design & Competition Selection	2-3
3	Feature Engineering: Pre-Match and In-Play Pipelines	2-3
4	Paper Reproductions (P1 & P2) with fidelity tables	3-4
5	Experimental Setup (splits, snapshotting, metrics, calibration)	1-2
6	Results: Models 1, 2, and 3 (full metric tables)	2-3
7	Calibration & Reliability Analysis (incl. per-phase for Model 3)	1-2
8A	Error & Failure Analysis (10-worst per task, with SHAP)	1-2
8B	In-Play Analysis: metric-vs-minute curves, SHAP timeline	1-2
8C	Imbalance & Resampling Discussion	1
8D	Compute & Scaling Analysis (kernel $O(n^2)$ section)	1
8E	Theory vs. Reality	1
9	Conclusion & Limitations	1
App. A	Hand-derivation of the P2 core equation	as needed
App. B	Reproducibility appendix (requirements, seeds, git log)	as needed

8.1 Results Table Format

For every model and task combination, report one unified results table. Bold the best value in each column, and always include the Dummy baseline row (and, for Task L, the frozen pre-match prediction row) so the reader can locate your models against the sanity floor. Never present only the single metric on which a model happens to look best.

8.2 Writing Standards

- Cite every dataset, library, and algorithm with a full bibliographic reference, including the StatsBomb attribution required by the data licence.
- Give every figure a caption and every table a title row, and refer to each by number in the text.
- Report negative results honestly. A model that fails to beat its pre-match prior in-play is a valid finding when you explain why.
- Use past tense for results ('Table 3 shows...') and present tense for methods ('We compute rolling form as...').

9. SUBMISSION CHECKLIST

Confirm every item below before submitting. Incomplete submissions cannot receive full marks.

- Full written report as a single PDF, maximum 25 pages excluding appendices.
- Reproducible, zipped code appendix with all scripts and notebooks and a pinned requirements.txt.
- The integration layer: documented code that builds your relational store from the raw StatsBomb JSON tree.
- Both feature pipelines (pre-match and in-play) as runnable, tested code with the time-t cut assertion.
- All three model deliverables trained and evaluated, with fixed random seeds.
- The complete metric suite for every model on every task, with no selective reporting.
- Calibration curves and reliability diagrams for all probabilistic outputs; per-phase calibration for Model 3.
- Metric-vs-match-minute curves for Model 3 against the frozen pre-match baseline.
- A 10-worst-prediction post-mortem per task, supported by SHAP explanations.
- SHAP global summary plots per model, local plots for the worst predictions, and one full-match SHAP timeline.
- A compute and memory table covering all models, and the kernel scaling plot.
- Exported git history (git log --oneline --all) showing genuine iterative development.
- A hand-derivation appendix for the P2 model's core update or objective equation.
- The odds integration: Football-Data.co.uk CSVs tagged to StatsBomb match_ids, with the alias mapping, per-season coverage rates, and the list of excluded matches.
- The de-vigged market baseline evaluated on the identical tagged test matches as every Task C model.
- TA sign-off confirmation for both P1 and P2, obtained at or before the corresponding mid defence.
- If attempting the bonus (Section 11): the running FastAPI service, the dashboard, and the latency evidence.

10. DEFENCE MILESTONES

Each group (1-2 people) completes three live defences with the teaching assistant: two mid-project defences that gate the two halves of the work, and one final defence built around a live demonstration. Every group member must be able to answer for every part of the project; in two-person groups the TA chooses who answers each question. Dates and booking are on the course portal.

Mid Defence 1 -- Data Paper (P1) & Data Pipeline

Held once your integration layer and feature pipelines exist in first working form. You present and defend two things.

- **Your P1 pick:** present the paper, argue why it clears the hard filters, and explain precisely how its method maps onto this dataset's real imperfections. Sign-off for P1 happens here.
- **Your data pipeline:** walk through the integration design (joins, keys, identity resolution), the formal problem definition for all three tasks (what one example is, what its label is), the temporal split, the snapshotting scheme, and your leakage defences, including the time- t cut assertion. Expect the TA to probe with what-if questions: what happens if a snapshot straddles half-time, how you handle matches without 360 data, why your competition selection is justified.

Mid Defence 2 -- Model Paper (P2) & Predictive Modeling

Held once your baseline models train end to end. You present and defend two things.

- **Your P2 pick:** present the paper, argue the filters, and walk through the core mathematics, including a sketch of the hand derivation destined for Appendix A. Sign-off for P2 happens here.
- **Your modeling approach:** justify the model suite configuration for each of the three deliverables, the hyperparameter budgets and search protocol, the calibration strategy, and how the in-play models are trained on the snapshot table without leaking across the time- t boundary or the match-level split. Expect design-consequence questions: predict what would change if snapshots were taken every minute instead of every five, or if SMOTE were applied to the snapshot table.

Final Defence -- Results & Live Online Case with SHAP

Held after submission. The centrepiece is a live, online walkthrough of one case, run in front of the TA on a match the TA selects from your held-out data.

- **Present the results:** summarise the final metric tables for all three models, the calibration story, and the headline findings from the error analysis.
- **Run the online case live:** for the selected match, run your pipeline to produce the pre-match prediction (Models 1 and 2), then step through the match with Model 3, showing the probability and margin estimates updating at successive snapshots.
- **Explain it with SHAP:** at each shown snapshot, present the SHAP attribution and narrate it: which features carry the pre-match prediction, and how the attributions shift as goals, cards, and momentum arrive. State whether the explanation is sensible and what it implies about the model's reliability.

Why this format: a model that only reports an aggregate score is impossible to trust or debug. The live case proves that your pipeline genuinely runs end to end on unseen input, that the in-play machinery respects

the time- t boundary in practice and not just on paper, and that you understand what your models key on well enough to narrate a single decision the way a real analyst must when a forecast is challenged. It is also the clearest possible evidence that the work is your own. Groups attempting the bonus (Section 11) demonstrate the live API and dashboard here as well.

11. BONUS: REAL-TIME PREDICTION SERVICE

This section is optional and carries bonus credit on top of the project grade. It turns your in-play model into what it would actually be in production: a low-latency prediction service driving a live dashboard. It is the most industry-realistic part of the project, and groups that attempt it must be able to present it running live at the final defence.

11.1 The API

Build a FastAPI service that wraps your trained Model 3 (and, for the pre-kickoff state, Models 1 and 2). It must expose a prediction endpoint that accepts the identifier of a match and a time t (or a payload of events up to t), assembles the snapshot feature vector through the same feature code used in training, and returns the current outcome probabilities and expected final margin as JSON.

- **Latency budget:** the endpoint must respond in under 200 milliseconds end to end on your demo machine. This forces real engineering: load the model once at startup, precompute and cache the pre-match vector and running in-play aggregates rather than rescanning the event file per request, and keep feature assembly incremental. Measure and report the latency distribution (p50 / p95 / p99) under repeated calls, not a single lucky timing.
- **Correctness first:** the API must reuse the training feature pipeline, not a reimplementation. A fast endpoint that computes features differently from training is a deployment bug, and the TA will check for it by comparing an API response against your offline prediction for the same snapshot.
- **Suggested stack:** FastAPI with uvicorn, a preloaded model object, and an in-memory state store per streamed match. No database is required.

11.2 The Dashboard

Build a small live dashboard that replays a real match from your held-out data as if it were happening now: a driver script feeds events to the service in match order, and the dashboard updates after each snapshot.

- **Must show:** the evolving outcome probabilities (three lines or a stacked area over match time), the expected final margin, the current score and key events (goals, cards) as markers, and the top SHAP attributions for the latest snapshot.
- **Keep it simple:** Streamlit, Plotly Dash, or a plain HTML page polling the API are all acceptable. The dashboard is judged on correctness and clarity, not visual polish.

11.3 Presenting It

At the final defence, start the service, launch the replay on a TA-selected held-out match, and narrate what the dashboard shows as the match unfolds: why the probabilities move when they move, what the SHAP panel says is driving the shift, and where the latency stands. Be prepared to send a hand-crafted request to the endpoint live and explain the response. Bonus credit is awarded only if the demonstration works in the room.

12. REFERENCES

The following foundational references cover the data source and standard tools used in this project. Supplement them with your own P1 and P2 citations and any additional sources referenced in your report.

- [1] StatsBomb / Hudl StatsBomb (2018-2025). StatsBomb Open Data. Available at: <https://github.com/statsbomb/open-data> (free for research and non-commercial use with attribution).
- [2] Football-Data.co.uk. Historical Football Results and Betting Odds Data. Available at: <https://www.football-data.co.uk> (per-league season CSVs; field key in notes.txt).
- [3] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321-357.
- [4] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-794.
- [5] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T. Y. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems*, 30, 3146-3154.
- [6] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- [7] Lundberg, S. M., & Lee, S. I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*, 30, 4765-4774.
- [8] Lundberg, S. M., et al. (2020). From Local Explanations to Global Understanding with Explainable AI for Trees. *Nature Machine Intelligence*, 2(1), 56-67.
- [9] Lemaitre, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning. *Journal of Machine Learning Research*, 18(17), 1-5.
- [10] Constantinou, A. C., & Fenton, N. E. (2012). Solving the Problem of Inadequate Scoring Rules for Assessing Probabilistic Football Forecast Models. *Journal of Quantitative Analysis in Sports*, 8(1).
- [11] Niculescu-Mizil, A., & Caruana, R. (2005). Predicting Good Probabilities with Supervised Learning. *Proceedings of the 22nd International Conference on Machine Learning*, 625-632.